ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO



BÁO CÁO ĐỒ ÁN HỌC PHẦN

PBL3: MẠNG MÁY TÍNH – NHÓM 7

ĐỀ TÀI:

PGP / OPENPGP

NGHIÊN CỨU VÀ MÔ PHỎNG CƠ CHẾ MÃ HÓA LAI (HYBRID ENCRYPTION) KẾT HỢP RSA VÀ AES-GCM

Giảng viên hướng dẫn: TS. Trần Thị Minh Hạnh

Sinh viên thực hiện (Nhóm 7):

STT	Họ và tên	MSSV
1	Vũ Duy Dương	106230007
2	Huỳnh Thiện Lộc	106230052
3	Phan Vĩnh Sơn	106230059

Đà Nẵng, năm 2026

MỤC LỤC

1. Giới thiệu **3**
1.1. Bối cảnh kỹ thuật . . . 3
1.2. Mục tiêu đề tài . . . 4
1.3. Phạm vi đề tài . . . 4
1.4. Phân công nhiệm vụ và đóng góp của các thành viên . . . 5

2. Các nghiên cứu và phương pháp liên quan **6**
2.1. Tiêu chuẩn mã hóa nâng cao AES và chế độ xác thực GCM . . . 6
2.2. Hệ mật mã bất đối xứng RSA và bài toán phân tích số nguyên . . . 6
2.3. Chuẩn bảo mật thư điện tử PGP và OpenPGP . . . 6
2.4. So sánh kỹ thuật và lý do kết hợp mã hóa lai . . . 6

3. Phương pháp thực hiện **8**
3.1. Quy trình Hybrid Encryption tổng thể . . . 8
3.2. Cơ sở số học của thuật toán RSA . . . 9
3.3. Cơ chế mã hóa xác thực AES-GCM (AEAD) . . . 10
3.4. Đóng gói khung truyền thông mạng và giao thức Socket TCP . . . 10

4. Thực nghiệm và kết quả **12**
4.1. Môi trường thực nghiệm và kiến trúc mã nguồn . . . 12
4.2. Ví dụ số học minh họa thuật toán RSA trên không gian nhỏ . . . 12
4.3. Kết quả thực nghiệm 3 kịch bản kiểm thử an toàn . . . 13
4.4. Thực nghiệm truyền thông qua Socket TCP giữa 2 máy mạng LAN . . . 15
4.5. Giao diện mô phỏng trực quan NiceGUI (Web Simulator) . . . 16

5. Kết luận **17**
5.1. Đánh giá kết quả đạt được . . . 17
5.2. Hạn chế và hướng phát triển tiếp theo . . . 17

TÀI LIỆU THAM KHẢO **18**

PHỤ LỤC **19**
Phụ lục A: Cài đặt giải thuật Euclid mở rộng và RSA số học . . . 19
Phụ lục B: Cài đặt AES-GCM và bộ đóng gói PGP Hybrid . . . 20
Phụ lục C: Cài đặt truyền thông Socket TCP giữa 2 máy mạng LAN . . . 21

Tóm tắt:
Trong truyền thông mạng mở, mật mã đối xứng AES (Advanced Encryption Standard) đạt tốc độ xử lý cao với dữ liệu lớn nhưng gặp bài toán phân phối khóa bí mật qua kênh không an toàn. Ngược lại, mật mã bất đối xứng RSA (Rivest–Shamir–Adleman) giải quyết trao đổi khóa bằng cặp khóa công khai / bí mật nhưng có chi phí lũy thừa mô-đun lớn, không phù hợp để mã hóa trực tiếp thông điệp dài.

Báo cáo trình bày kiến trúc Mã hóa lai (Hybrid Encryption) theo nguyên lý PGP/OpenPGP (RFC 4880, RFC 9580): dùng AES-128-GCM mã hóa dữ liệu kèm xác thực toàn vẹn GHASH, và dùng RSA số học (modulus 256-bit, sinh nguyên tố Miller–Rabin) để bảo vệ khóa phiên dùng một lần (Session Key). Hệ thống được triển khai qua Socket TCP giữa Client (Alice) và Server (Bob) với cơ chế đóng khung chống phân mảnh luồng byte, cùng ứng dụng mô phỏng tương tác NiceGUI. Thực nghiệm qua 3 kịch bản kiểm thử cho thấy hệ thống khôi phục chính xác dữ liệu gốc và phát hiện, từ chối xử lý khi có can thiệp sửa đổi bit trên đường truyền.

Từ khóa: PGP, OpenPGP, RSA, AES-GCM, Mã hóa lai (Hybrid Encryption), Socket TCP, Xác thực toàn vẹn, GHASH, Miller–Rabin, Euclid mở rộng.

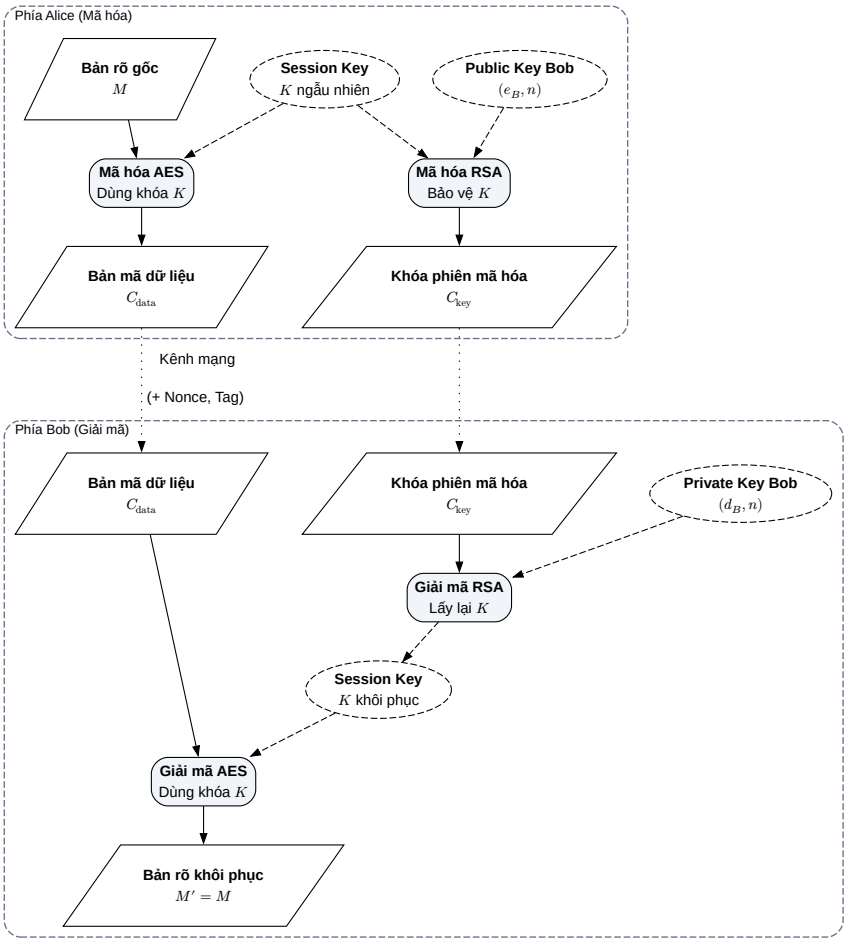
1. Giới thiệu

1.1. Bối cảnh kỹ thuật

Theo lý thuyết an toàn mạng [1], một hệ thống truyền thông tin cậy cần bốn thuộc tính: (1) **Tính bí mật (Confidentiality)** – ngăn bên không có thẩm quyền đọc nội dung; (2) **Tính toàn vẹn (Integrity)** – bảo đảm dữ liệu không bị sửa đổi, chèn hoặc xóa trên đường truyền; (3) **Tính xác thực (Authentication)** – xác minh danh tính bên gửi; và (4) **Tính chống chối bỏ (Non-repudiation)** – bên gửi không thể phủ nhận thông điệp đã tạo.

Hai trường phái mật mã chính giải quyết các yêu cầu này là mật mã đối xứng và bất đối xứng. AES (Advanced Encryption Standard) [2] cho tốc độ mã hóa/giải mã cao nhờ mạng hoán vị – thay thế (SPN) tối ưu phần cứng, nhưng gặp khó khăn trong việc phân phối khóa bí mật qua mạng công khai. Ngược lại, RSA (Rivest, Shamir, Adleman, 1978) [3] giải quyết bài toán trao đổi khóa bằng cặp khóa công khai / bí mật, nhưng phép lũy thừa mô-đun trên số nguyên lớn khiến RSA chậm hơn AES từ 100 đến 1000 lần, không phù hợp để mã hóa trực tiếp thông điệp dài.

Chuẩn PGP và OpenPGP (RFC 4880 [4], RFC 9580 [5]) kết hợp hai phương pháp qua kiến trúc **Mã hóa lai (Hybrid Encryption)**: dùng AES mã hóa bản rõ bằng khóa phiên dùng một lần (Session Key), rồi dùng RSA mã hóa khóa phiên đó bằng khóa công khai người nhận. Quy trình giữa Alice (gửi) và Bob (nhận) được minh họa trong Hình 1: bản rõ M được mã hóa đối xứng bằng khóa phiên K thành C_{data} (kèm Nonce và Authentication Tag), khóa K được mã hóa bất đối xứng bằng Public Key (e_B, n) thành C_{key}; phía Bob dùng Private Key (d_B, n) giải mã lấy lại K để khôi phục bản rõ $M' = M$.



Hình 1: Sơ đồ nguyên lý mã hóa và giải mã lai RSA – AES khép kín trong mô hình PGP [6]

1.2. Mục tiêu đề tài

Đề tài PBL3: Mạng máy tính (Khoa Điện tử và Trí tuệ Nhân tạo, Trường Đại học Bách Khoa – ĐHĐN) hướng tới các mục tiêu sau:

1. **Cơ sở số học và mật mã:** Nắm vững lý thuyết RSA (Euclid mở rộng, định lý Euler, kiểm tra nguyên tố Miller–Rabin) và mã hóa xác thực AES-GCM (Galois/Counter Mode).
2. **Cài đặt RSA thuần Python:** Tự lập trình giải thuật RSA không dùng thư viện đen (black-box) để minh họa trực tiếp sinh khóa và lũy thừa mô-đun.
3. **Tích hợp luồng lai PGP:** Kết nối module RSA với AES-GCM từ khâu mã hóa khóa phiên, đóng gói khung tin đến giải mã và kiểm thực.
4. **Truyền thông Socket TCP:** Xây dựng hệ thống Client–Server qua TCP Socket có đóng khung độ dài (Length-Prefixed Framing) chống phân mảnh luồng byte giữa 2 máy mạng LAN.
5. **Mô phỏng và kiểm thử:** Phát triển giao diện web NiceGUI gồm hai chế độ (Tích hợp luồng truyền thông và Chi tiết từng khối thuật toán), kèm bộ kịch bản kiểm thử tấn công sửa bit.

1.3. Phạm vi đề tài

- **Phạm vi thực hiện:** Cài đặt Textbook RSA dạng số học (modulus 256-bit ở demo thuật toán, 512-bit ở truyền thông Socket, đủ chứa khóa AES 128-bit); dùng thư viện chuẩn AES-GCM với Nonce 12 bytes và Tag 16 bytes; đóng gói JSON Envelope truyền qua TCP Socket

cổng 5000; đối chiếu SHA-256 fingerprint của Public Key; xây dựng Web Simulator NiceGUI. Hệ thống bảo đảm **Tính bí mật**, **Tính toàn vẹn** và xác thực thủ công khóa công khai qua SHA-256 fingerprint.

- **Phạm vi chưa thực hiện:** Chưa triển khai đệm RSA-OAEP theo RFC 8017 [7] (chống tấn công Coppersmith [8]); chưa tích hợp Chữ ký số (Digital Signature) cho tính xác thực người gửi và chống chối bỏ; chưa xây dựng Web of Trust và gói tin nhị phân đa tầng đầy đủ theo RFC 4880 [4].

1.4. Phân công nhiệm vụ và đóng góp của các thành viên

Phân công công việc của 3 thành viên được tổng hợp trong Bảng 1.

Tên thành viên	Nhiệm vụ phụ trách	Hoàn thành
Vũ Duy Dương (`106230007`)	Trưởng nhóm; thiết kế kiến trúc Hybrid Encryption; cài đặt RSA thuần (Miller–Rabin, Euclid mở rộng); phát triển Web Simulator NiceGUI; tổng hợp báo cáo.	100%
Huỳnh Thiện Lộc (`106230052`)	Nghiên cứu AES-GCM (NIST SP 800-38D) và hàm băm GHASH trên $\mathrm{GF}(2^{128})$; thiết kế và thực thi 3 kịch bản kiểm thử bảo mật và toàn vẹn.	100%
Phan Vĩnh Sơn (`106230059`)	Nghiên cứu TCP Socket; lập trình Client–Server (Alice/Bob) qua mạng LAN; cài đặt đóng khung tiền tố độ dài chống phân mảnh và đối chiếu SHA-256 fingerprint.	100%

Bảng 1: Phân công nhiệm vụ của nhóm.

2. Các nghiên cứu và phương pháp liên quan

2.1. Tiêu chuẩn mã hóa nâng cao AES và chế độ xác thực GCM

Năm 2001, NIST ban hành Tiêu chuẩn Mã hóa Nâng cao AES (FIPS PUB 197 [2]) thay thế DES. AES là mật mã khối kích thước 128 bit, hỗ trợ khóa 128, 192 và 256 bit, vận hành theo cấu trúc mạng hoán vị – thay thế (SPN) qua 10 đến 14 vòng lặp với bốn phép toán: `SubBytes` (S-box), `ShiftRows`, `MixColumns` trên trường Galois $\mathrm{GF}(2^8)$ và `AddRoundKey` (XOR).

Các chế độ truyền thống như CBC hay ECB chỉ bảo đảm tính bí mật, không tự phát hiện sửa đổi bản mã. Để khắc phục, chế độ Galois/Counter Mode (GCM) do McGrew và Viega đề xuất năm 2004 [9] được NIST chuẩn hóa trong SP 800-38D [10]. Thuộc họ AEAD (Authenticated Encryption with Associated Data), AES-GCM kết hợp mã hóa dòng Counter Mode (CTR) song song với hàm băm vạn năng GHASH trên $\mathrm{GF}(2^{128})$, sinh Authentication Tag 128-bit để phát hiện mọi thay đổi bit trên đường truyền.

2.2. Hệ mật mã bất đối xứng RSA và bài toán phân tích số nguyên

Khái niệm mật mã khóa công khai do Diffie và Hellman đề xuất năm 1976 [11] cho phép trao đổi thông tin mật mà không cần thỏa thuận khóa trước. Trên cơ sở đó, Rivest, Shamir và Adleman công bố thuật toán RSA năm 1977 tại MIT [3]. Độ an toàn của RSA dựa trên bài toán phân tích số nguyên lớn (Integer Factorization Problem): tính $n = p \cdot q$ từ hai số nguyên tố p, q rất nhanh, nhưng tìm ngược lại p, q từ n chưa có thuật toán thời gian đa thức trên máy tính cổ điển khi $n \geq 2048$ bit.

Chuẩn PKCS 1 / RFC 8017 [7] quy định cấu trúc khóa và cơ chế toán học của RSA: công khai (e, n) và giữ bí mật d. Nhờ tính đối ngẫu toán học, dữ liệu mã hóa bằng Public Key (e, n) chỉ giải mã được bằng Private Key (d, n) (bảo mật), trong khi dữ liệu xử lý bằng Private Key (d, n) có thể kiểm chứng bằng Public Key (e, n) (chữ ký số).

2.3. Chuẩn bảo mật thư điện tử PGP và OpenPGP

Năm 1991, Phil Zimmermann phát triển PGP (Pretty Good Privacy), sau được IETF chuẩn hóa thành OpenPGP qua RFC 4880 [4] và RFC 9580 [5]. OpenPGP tích hợp các chuẩn mật mã sẵn có: mã hóa đối xứng (AES, Triple-DES, IDEA) cho nội dung thư; mã hóa bất đối xứng (RSA, ElGamal, ECC) cho khóa phiên; và hàm băm (SHA-256, SHA-512) kết hợp RSA cho chữ ký số. OpenPGP cũng sử dụng mô hình Web of Trust, cho phép người dùng ký chéo khóa công khai thay vì phụ thuộc vào CA tập trung.

2.4. So sánh kỹ thuật và lý do kết hợp mã hóa lai

Bảng 2 đối chiếu đặc tính kỹ thuật giữa AES và RSA, cho thấy mỗi hệ mật có ưu điểm và hạn chế riêng.

Tiêu chí so sánh	AES-128-GCM (Đối xứng)	RSA-256 / 512 (Bất đối xứng)
Số lượng khóa	1 khóa phiên bí mật K chung cho cả hai đầu.	Cặp khóa: Public Key (e, n) và Private Key (d, n).
Tốc độ & Chi phí	Rất nhanh; chi phí phần cứng thấp (XOR, tra bảng S-box).	Chậm; đòi hỏi nhân và lũy thừa mô-đun trên số nguyên lớn.
Xử lý dữ liệu lớn	Phù hợp cho tệp tin, video và luồng mạng lớn.	Không phù hợp; chỉ mã hóa được giá trị nhỏ hơn modulus n.
Phân phối khóa	Khó; cần kênh truyền an toàn sẵn để chia sẻ K.	Dễ; Public Key phát tán tự do qua mạng mở.
Toàn vẹn & Chống sửa đổi	Tích hợp sẵn qua GHASH và Authentication Tag 128-bit.	Không tự kiểm thực; cần chữ ký số (RSA-PSS + băm) và RSA-OAEP cho IND-CCA2.

Bảng 2: So sánh đặc tính kỹ thuật giữa hệ mật mã đối xứng AES và bất đối xứng RSA.

Giải pháp **Mã hóa lai (Hybrid Encryption)** kết hợp cả hai theo nguyên lý: AES mã hóa dữ liệu, RSA bảo vệ khóa phiên. Gói tin PGP Hybrid được biểu diễn bởi:

$$\mathrm{PGP_Packet} = \left\{ \mathrm{RSA}_{e_B}(K), \quad \mathrm{AES}_K(M) \right\}$$

Trong đó e_B là khóa công khai của Bob, K là khóa phiên ngẫu nhiên dùng một lần do Alice tạo, và M là bản rõ. Mô hình này kết hợp tốc độ của AES với khả năng phân phối khóa của RSA (kích thước RSA 256/512-bit trong đồ án phục vụ mô phỏng học thuật; triển khai thực tế cần RSA ≥ 2048-bit kèm đệm RSA-OAEP).

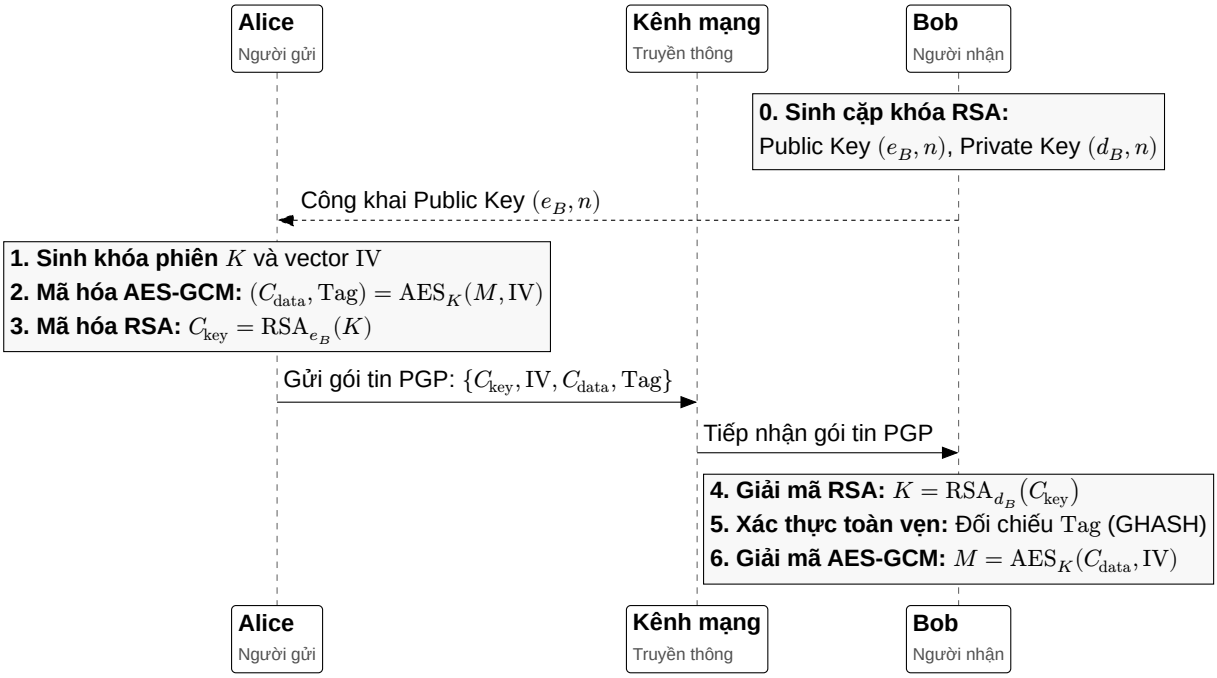
3. Phương pháp thực hiện

3.1. Quy trình Hybrid Encryption tổng thể

Luồng truyền thông giữa Alice (gửi) và Bob (nhận) gồm 5 giai đoạn tuần tự như mô tả trong Bảng 3 và sơ đồ tương tác ở Hình 2.

Giai đoạn	Phía Alice (Người gửi)	Phía Bob (Người nhận)
GĐ 0: Khởi tạo danh tính	Nhận Public Key (e, n) của Bob qua mạng hoặc tệp cấu hình.	Sinh cặp khóa RSA (e, n) và (d, n); công khai (e, n).
GĐ 1: Sinh khóa phiên K	Dùng CSPRNG (`os.urandom`) sinh Session Key K (16 bytes = 128-bit).	Chờ gói tin từ phía gửi qua socket mạng.
GĐ 2: Mã hóa dữ liệu & khóa	• AES-GCM mã hóa M bằng $K \rightarrow$ `ciphertext`, `nonce`, `auth_tag`. - RSA mã hóa K bằng $(e, n) \rightarrow$ `enc_session_key`.	Không thực hiện.
GĐ 3: Đóng gói & Truyền tải	Đóng gói JSON Envelope 4 trường mật mã, thêm tiền tố 4-byte độ dài gửi qua TCP.	Đọc 4-byte độ dài, nhận đủ gói JSON Envelope vào bộ nhớ đệm.
GĐ 4: Giải mã & Kiểm thực	Chờ phản hồi kết quả từ máy nhận.	• Dùng d giải RSA: $K' = (C_{\text{key}})^d \bmod n$. - Kiểm thực `auth_tag` qua GHASH; khớp thì giải mã ra M'.

Bảng 3: Quy trình thực hiện mã hóa lai giữa phía gửi (Alice) và phía nhận (Bob).



Hình 2: Sơ đồ tuần tự quy trình truyền thông mã hóa lai PGP giữa Alice và Bob

3.2. Cơ sở số học của thuật toán RSA

Thuật toán RSA số học thuần túy gồm các bước sau:

1. **Kiểm tra nguyên tố xác suất Miller–Rabin:** Sinh hai số nguyên tố p, q (128-bit cho RSA-256 ở demo thuật toán; 256-bit cho RSA-512 ở giao tiếp Socket) bằng thuật toán Miller–Rabin [12] với $k = 20$ vòng lặp (trong `algorithms/rsa.py`). Xác suất hợp số lọt qua nhỏ hơn $\left(\frac{1}{4}\right)^{20} \approx 9.09 \times 10^{-13}$.
2. **Modulus và hàm Euler:** Tính $n = p \cdot q$ (256 hoặc 512 bit) và $\varphi(n) = (p-1)(q-1)$.
3. **Số mũ công khai Fermat $e = 65537$:** Chọn $e = 2^{16} + 1$ (số nguyên tố Fermat F_4, dạng nhị phân 10000000000000001_2 chỉ có 2 bit 1), giúp phép lũy thừa theo mô-đun chỉ cần 16 phép bình phương và 1 phép nhân.
4. **Nghịch đảo mô-đun (Euclid mở rộng):** Tìm khóa bí mật d từ $e \cdot d \equiv 1 (\text{mod}\, \varphi(n))$, tương đương phương trình Bézout $e \cdot d + \varphi(n) \cdot y = 1$, với độ phức tạp $O(\log(\varphi(n)))$.

Chứng minh tính đúng đắn dựa trên Định lý Euler:

Phía Bob tính:

$$K' = \left(C_{\text{key}}\right)^d \bmod n = \left((K^e) \bmod n\right)^d \bmod n = K^{e \cdot d} \bmod n$$

Vì $e \cdot d \equiv 1 (\text{mod}\, \varphi(n))$, tồn tại $k \in \mathbb{Z}$ sao cho $e \cdot d = 1 + k \cdot \varphi(n)$. Khi $\gcd(K, n) = 1$, theo Định lý Euler $K^{\varphi(n)} \equiv 1 (\text{mod}\, n)$:

$$K' \equiv K^{1 + k \cdot \varphi(n)} \equiv K \cdot \left(K^{\varphi(n)}\right)^k \equiv K \cdot 1^k \equiv K (\text{mod}\, n)$$

Nếu $\gcd(K, n) \neq 1$ (K là bội của p hoặc q), kết hợp Định lý Fermat nhỏ trên p, q và Định lý Số dư Trung Hoa (CRT) vẫn cho $K^{e \cdot d} \equiv K (\text{mod}\, p)$ và $K^{e \cdot d} \equiv K (\text{mod}\, q)$, suy ra $K^{e \cdot d} \equiv K (\text{mod}\, n)$. Do K (128-bit) thỏa $K < 2^{128} < n$, giá trị K' trùng khớp với K ban đầu.

3.3. Cơ chế mã hóa xác thực AES-GCM (AEAD)

Theo chuẩn NIST SP 800-38D [10], AES-GCM thực hiện đồng thời mã hóa và xác thực:

1. **Mã hóa dòng Counter Mode (CTR):** Với Nonce 96-bit (12 bytes), khối đếm ban đầu là $J_0 = \mathrm{CB}_0 = \mathrm{Nonce} \parallel 00000001_{16}$. Bản rõ M được chia thành các khối 16 bytes $P_1, P_2, ..., P_m$; các khối đếm tăng dần qua phép tăng 32-bit cuối $\mathrm{CB}_i = \mathrm{inc}_{32}(\mathrm{CB}_{i-1})$ ($i \geq 1$, tức $\mathrm{CB}_1 = \mathrm{Nonce} \parallel 00000002_{16}$, $\mathrm{CB}_2 = \mathrm{Nonce} \parallel 00000003_{16}$, ...). Bản mã được tính bằng:

$$C_i = P_i \oplus \mathrm{AES}_K(\mathrm{CB}_i)$$

   Cơ chế này không cần padding và hỗ trợ tính toán song song.
2. **Xác thực toàn vẹn bằng GHASH:** Khóa băm $H = \mathrm{AES}_K(0^{128})$. Phép nhân trên trường hữu hạn $\mathrm{GF}(2^{128})$ sử dụng đa thức bất khả quy:

$$f(x) = x^{128} + x^7 + x^2 + x + 1$$

   Khối bản mã cuối C_m nếu chưa đủ 128 bit được đệm bit 0 về bên phải. Giá trị băm tích lũy là $X_i = (X_{i-1} \oplus C_i) \cdot H$ ($X_0 = 0^{128}$). Cuối cùng, GHASH nhân thêm khối 128-bit độ dài bit của dữ liệu bổ sung A và bản mã C ($\mathrm{len}_{64}(A) \parallel \mathrm{len}_{64}(C)$, với $A = \varnothing$), rồi XOR với $\mathrm{AES}_K(\mathrm{CB}_0)$ để tạo Authentication Tag:

$$\mathrm{Tag} = ((X_m \oplus (\mathrm{len}_{64}(A) \parallel \mathrm{len}_{64}(C))) \cdot H) \oplus \mathrm{AES}_K(\mathrm{CB}_0)$$

Nhờ tính khuếch tán trên $\mathrm{GF}(2^{128})$, mọi thay đổi dù chỉ 1 bit trong bản mã đều làm sai lệch giá trị GHASH tính lại tại Bob. Khi `"Tag"_"computed" != "Tag"_"received"`, hệ thống ném ngoại lệ `ValueError: MAC check failed` và hủy gói tin ngay tại tầng giải mã.

3.4. Đóng gói khung truyền thông mạng và giao thức Socket TCP

TCP truyền dữ liệu dạng luồng byte (byte stream), không giữ ranh giới thông điệp nên dễ xảy ra phân mảnh hoặc dính gói. Hệ thống khắc phục bằng **đóng khung tiền tố độ dài (Length-Prefixed Framing)**: mỗi gói tin đều gắn tiền tố 4-byte số nguyên không dấu Big-Endian (`!I`) chỉ kích thước payload JSON UTF-8 đi kèm (và kiểm tra giới hạn kích thước tối đa ở phía nhận để chống tràn bộ nhớ).

Cấu trúc JSON Envelope được mô tả trong Bảng 4. Ở tầng Socket (`PGP_RSA_AES_Socket_3_files/`), `enc_session_key` truyền dưới dạng số nguyên lớn (hoặc chuỗi hex ở CLI demo), còn `nonce` , `ciphertext` , `auth_tag` được mã hóa Base64 (gói tin gửi lên server có thêm `"action": "SEND_MESSAGE"`).

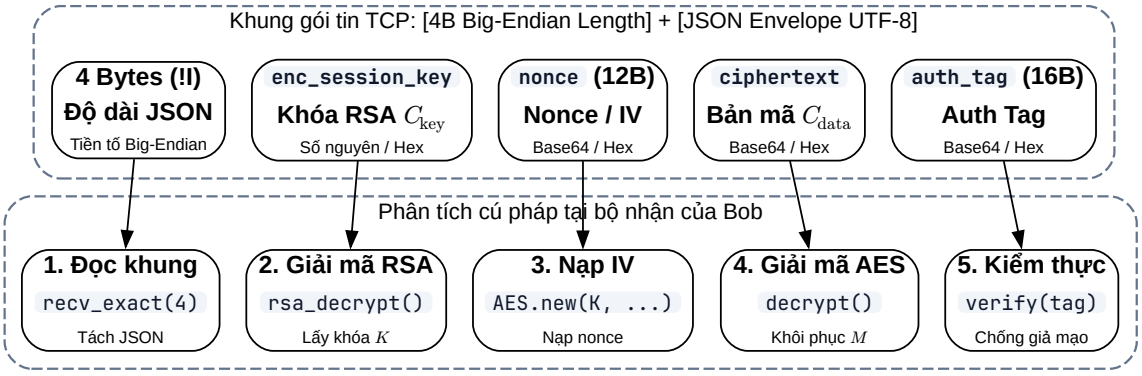
Tên trường JSON	Kích thước gốc	Định dạng truyền tải	Mô tả vai trò mật mã
`enc_session_key`	256 / 512 bit	Số nguyên lớn / Hex	Khóa phiên K (128-bit) đã mã hóa bằng Public Key của Bob qua RSA.
`nonce`	12 bytes (96 bit)	Base64 / Hex	Vector khởi tạo ngẫu nhiên dùng một lần cho AES-GCM.
`ciphertext`	Biến thiên ($\|M\|$ bytes)	Base64 / Hex	Bản mã của M sinh bởi AES CTR Mode.
`auth_tag`	16 bytes (128 bit)	Base64 / Hex	Mã xác thực sinh bởi hàm băm GHASH trên $\mathrm{GF}(2^{128})$.

Bảng 4: Cấu trúc trường dữ liệu của gói tin PGP Hybrid qua mạng.

Để chống tấn công kẻ đứng giữa (Man-in-the-Middle) giả mạo khóa công khai, hệ thống tạo dấu vân tay SHA-256 (64 ký tự hex) cho Public Key của Bob để hai bên đối chiếu trước khi gửi:

$$\mathrm{fingerprint}(\mathrm{pub_key}) = \mathrm{SHA256}(\mathrm{RSA:} \parallel e \parallel : \parallel n)$$

Quy cách đóng gói khung và luồng phân tích cú pháp tại Bob được minh họa trong Hình 3.



Hình 3: Sơ đồ cấu trúc khung gói tin (Packet Framing) và luồng phân tích cú pháp tại bộ nhận của Bob

4. Thực nghiệm và kết quả

4.1. Môi trường thực nghiệm và kiến trúc mã nguồn

Hệ thống tổ chức lõi thuật toán tại `algorithms/` dùng chung cho giao diện NiceGUI, bên cạnh các kịch bản CLI và Socket TCP:

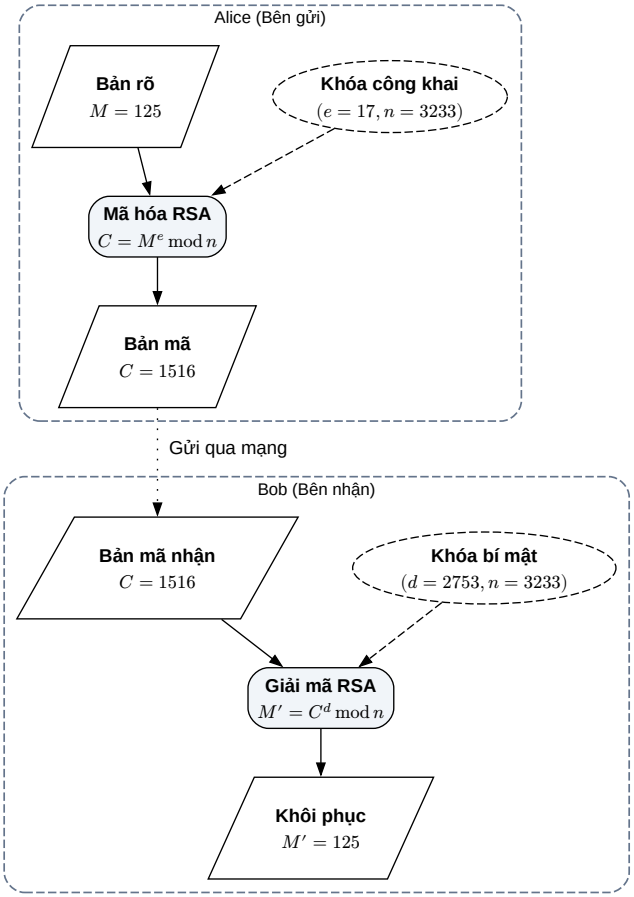
- `algorithms/euclid.py`: Thuật toán Euclid mở rộng (`extended_gcd`) và nghịch đảo mô-đun (`mod_inverse`).
- `algorithms/rsa.py`: Kiểm tra nguyên tố Miller–Rabin (`is_prime` , $k = 20$), sinh số nguyên tố (`generate_prime`), sinh khóa (`generate_rsa_keys`), mã hóa (`rsa_encrypt`) và giải mã (`rsa_decrypt`).
- `algorithms/aes_gcm.py`: Hàm `aes_gcm_encrypt` và `aes_gcm_decrypt` sử dụng thư viện PyCryptodome.
- `algorithms/hybrid_pgp.py` : Đóng gói và giải mã luồng lai PGP (`alice_encrypt_package` , `bob_decrypt_package`).
- `PGP_RSA_AES_Socket_3_files/` : Gồm `pgp_crypto.py` , `server_bob.py` (TCP Server, RSA 512-bit) và `client_alice.py` (TCP Client) truyền thông qua mạng LAN.
- `src/hybrid_demo.py` : Kịch bản CLI chạy 3 kịch bản kiểm thử bảo mật (kết hợp `src/rsa.py`).
- `src/gui_nicegui.py` : Ứng dụng web mô phỏng tương tác hai chế độ.

4.2. Ví dụ số học minh họa thuật toán RSA trên không gian nhỏ

Để kiểm chứng `algorithms/rsa.py` bằng tính toán thủ công, Bảng 5 và Hình 4 trình bày ví dụ trên không gian số nhỏ.

Bước tính toán	Công thức số học áp dụng	Giá trị kết quả
1. Chọn số nguyên tố	Chọn hai số nguyên tố p, q	$p = 61, \quad q = 53$
2. Tính Modulus	$n = p \cdot q = 61 \times 53$	$n = 3233$
3. Tính hàm Euler	$\varphi(n) = (p-1)(q-1) = 60 \times 52$	$\varphi(n) = 3120$
4. Chọn khóa công khai	$\gcd(e, \varphi(n)) = 1, \quad 1 < e < \varphi(n)$	$e = 17$
5. Tìm khóa bí mật	$17 \cdot d \equiv 1 (\mathrm{mod}\, 3120)$ qua Euclid mở rộng	$d = 2753$ (vì $17 \times 2753 = 46801 = 15 \times 3120 + 1$)
6. Thử nghiệm mã hóa / giải mã	Bản rõ $M = 125$ Mã hóa: $C = 125^{17} \bmod 3233$ Giải mã: $M' = 1516^{2753} \bmod 3233$	Bản mã $C = 1516$ Bản rõ khôi phục $M' = 125$ (Trùng khớp 100%)

Bảng 5: Tham số thực nghiệm tính toán số học trên không gian nhỏ của RSA.



Hình 4: Sơ đồ chu trình xử lý dữ liệu và khóa trong ví dụ số học RSA giữa Alice và Bob

4.3. Kết quả thực nghiệm 3 kịch bản kiểm thử an toàn

Bảng 6 tổng hợp kết quả 3 kịch bản kiểm thử trong `src/hybrid_demo.py`.

Kịch bản	Mô tả thực nghiệm	Cơ chế phản ứng của hệ thống	Kết luận
Test Case 01: Truyền thông hợp lệ	Bob dùng đúng Private Key (d, n) để giải mã.	RSA khôi phục 16 bytes Session Key K; AES-GCM khớp Tag GHASH và giải mã 100% bản rõ.	**PASSED** Khôi phục chính xác 100%.
Test Case 02: Tấn công can thiệp sửa bit (MITM)	Đảo 1 byte trong Ciphertext ($C_{\text{data}[0]} \oplus 0\text{xFF}$) trên đường truyền.	Khóa K khôi phục đúng, nhưng GHASH phát hiện Tag sai khác; ném `ValueError: MAC check failed` và hủy gói tin.	**PASSED** Chặn tấn công sửa đổi bản mã.
Test Case 03: Tấn công nghe lén bằng sai khóa	Eve dùng Private Key khác không hợp lệ để giải mã gói tin.	RSA giải ra Session Key K' sai; AES-GCM đối chiếu Tag thất bại và từ chối xuất dữ liệu.	**PASSED** Ngăn truy cập trái phép.

Bảng 6: Tổng hợp kết quả thực nghiệm 3 kịch bản kiểm thử bảo mật và toàn vẹn.

Nhật ký thực thi thực tế khi chạy `python3 src/hybrid_demo.py`:

```
Plain Text
1  ================================================================
2  DEMO PGP HYBRID ENCRYPTION (RSA + AES-GCM)
3  ================================================================
4
5  [ALICE] Thông điệp gốc: "Báo cáo tiến độ Nhóm 7: PGP Hybrid Encryption hoạt động chính xác!"
6  [ALICE] Mã hóa hoàn tất:
7    + Encrypted Session Key (RSA) : 0x8c47f6f87ccc179734ec19b039d026691e...
8    + Nonce (AES-GCM)             : 87e86fc1f28aaaa24fc05fff66b1185e
9    + Ciphertext (AES)            : 721796208643fdf15c2393663cd9990281f5c915525f810a... (81 bytes)
10   + Authentication Tag          : 04a4f16dfcb8b1f9e3a5e8825b17b0c9
11
12 ----------------------------------------------------------------
13 TEST CASE 01: Bob dùng đúng Private Key của mình
14 [BOB] Giải mã thành công: "Báo cáo tiến độ Nhóm 7: PGP Hybrid Encryption hoạt động chính xác!"
15 >>> KẾT QUẢ: [PASSED] Khôi phục chính xác 100% dữ liệu.
16
17 ----------------------------------------------------------------
18 TEST CASE 02: Kẻ tấn công sửa đổi 1 byte trong Ciphertext
19 [BOB] CẢNH BÁO AN TOÀN: Phát hiện dữ liệu bị can thiệp!
20       Chi tiết ngoại lệ: MAC check failed (MAC / Authentication Tag không khớp)
21 >>> KẾT QUẢ: [PASSED] Chặn thành công tấn công sửa đổi bản mã.
22
```

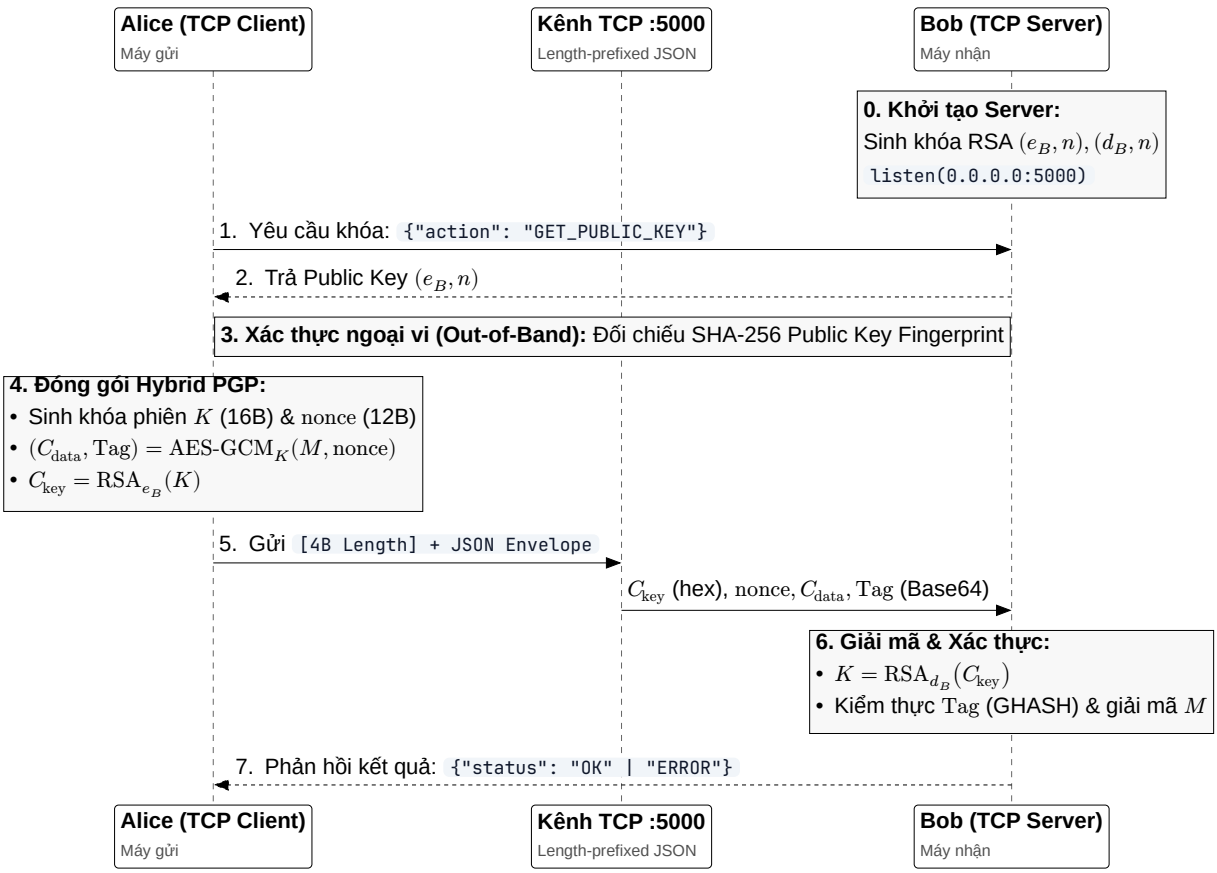
```
23 ----------------------------------------------------------------
24 TEST CASE 03: Giải mã bằng Private Key không hợp lệ (Khóa của Eve)
25 [EVE] Thất bại: Giải mã RSA ra Session Key sai -> AES-GCM từ chối xác thực.
26       Chi tiết ngoại lệ: MAC check failed (Sai khóa phiên dẫn đến kiểm thực thất
         bại)
27 >>> KẾT QUẢ: [PASSED] Từ chối truy cập trái phép khi sai khóa.
28 ================================================================
```

4.4. Thực nghiệm truyền thông qua Socket TCP giữa 2 máy mạng LAN

Thử nghiệm truyền thông được thực hiện giữa hai máy tính trong cùng mạng Wi-Fi cục bộ:

- **Máy B (Bob – Server):** Chạy `server_bob.py`, sinh cặp khóa RSA 512-bit, lắng nghe tại `0.0.0.0:5000` và hiển thị IP LAN (ví dụ `192.168.1.3`) cùng SHA-256 fingerprint của Public Key.
- **Máy A (Alice – Client):** Chạy `client_alice.py 192.168.1.3`, gửi lệnh `GET_PUBLIC_KEY` để lấy Public Key của Bob và đối chiếu SHA-256 fingerprint. Alice nhập thông điệp và chọn 1 trong 3 chế độ gửi (gửi chuẩn, giả lập sửa Ciphertext, giả lập sai khóa phiên). Độ trễ truyền nhận qua mạng LAN dưới 15 ms; phía Bob hiển thị bản rõ giải mã hoặc cảnh báo khi phát hiện can thiệp.

Luồng bắt tay trao đổi khóa và truyền nhận bản tin qua TCP Socket được minh họa trong Hình 5.



Hình 5: Sơ đồ tuần tự mô phỏng PGP Hybrid qua Socket TCP giữa máy Alice và máy Bob

4.5. Giao diện mô phỏng trực quan NiceGUI (Web Simulator)

Ứng dụng Web Simulator xây dựng bằng NiceGUI (`http://localhost:8080`) cung cấp hai chế độ mô phỏng:

1. **Chế độ "Mô Phỏng PGP (Tích Hợp)":** Trình bày luồng dữ liệu theo 3 cột thực thể: Alice (Client) $\rightarrow$ Kênh Mạng TCP $\rightarrow$ Bob (Server), gồm sơ đồ 8 khối dữ liệu động theo thời gian thực và bảng console log ghi lại từng bước xử lý.
2. **Chế độ "Chi Tiết Từng Khối Thuật Toán":** Trình bày 5 khối tuần tự từ trên xuống: Khối 1 (Tạo khóa RSA Bob) $\rightarrow$ Khối 2 (Mã hóa AES-GCM Alice) $\rightarrow$ Khối 3 (Mã hóa RSA khóa K) $\rightarrow$ Khối 4 (Giải mã RSA khóa K) $\rightarrow$ Khối 5 (Giải mã AES-GCM khôi phục M). Luồng dữ liệu truyền trực tiếp giá trị số nguyên 128-bit của khóa phiên K thật xuyên suốt từ Khối 2 đến Khối 5 để khôi phục thông điệp gốc.

5. Kết luận

5.1. Đánh giá kết quả đạt được

Đề tài đã hoàn thành các mục tiêu đề ra:

- **Về mặt học thuật:** Nắm vững cơ sở toán học của RSA và AES-GCM; chứng minh tính đúng đắn số học của giải mã RSA qua Định lý Euler (kèm CRT) và cơ chế kiểm thực của GHASH trên $\mathrm{GF}(2^{128})$.
- **Về mặt kỹ thuật:** Tự cài đặt RSA thuần Python (Miller–Rabin, Euclid mở rộng); tích hợp luồng mã hóa lai PGP; xử lý đóng khung độ dài (Length-Prefixed Framing) chống phân mảnh byte stream trên TCP Socket giữa 2 máy LAN.
- **Về mặt ứng dụng:** Xây dựng Web Simulator NiceGUI hai chế độ, hỗ trợ chạy tuần tự và giả lập tấn công sửa bit để kiểm chứng cơ chế mã hóa lai.

5.2. Hạn chế và hướng phát triển tiếp theo

Các hạn chế hiện tại và lộ trình mở rộng được tóm tắt trong Bảng 7.

Giai đoạn	Mục tiêu & Hạng mục kỹ thuật	Kết quả dự kiến đạt được
Giai đoạn 1	Tích hợp Chữ ký số điện tử (Digital Signature)	Kết hợp RSA với SHA-256 để ký bản rõ tại Alice và kiểm chứng tại Bob, bảo đảm xác thực người gửi và chống chối bỏ.
Giai đoạn 2	Nâng cấp bảo mật RSA theo chuẩn RFC 8017 [7]	Nâng khóa lên 2048-bit và áp dụng đệm RSA-OAEP nhằm khắc phục điểm yếu của Textbook RSA trước tấn công Coppersmith [8].
Giai đoạn 3	Mở rộng kiểm thử tệp tin dung lượng lớn	Thử nghiệm mã hóa và truyền nhận tệp PDF, hình ảnh, tệp nhị phân dung lượng từ vài chục MB đến hàng GB.
Giai đoạn 4	Đo đạc Benchmark hiệu năng	So sánh thời gian thực thi và mức tiêu thụ CPU/RAM giữa mã hóa thuần RSA với Mã hóa lai Hybrid Encryption.

Bảng 7: Kế hoạch hoàn thiện và mở rộng đề tài.

TÀI LIỆU THAM KHẢO

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th a.b. Pearson, 2017.

[2] National Institute of Standards and Technology, “Advanced Encryption Standard (AES)”, Federal Information Processing Standards Publication 197, 2001.

[3] R. L. Rivest, A. Shamir, và L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”, *Communications of the ACM*, tập 21, số 2, tr 120–126, 1978.

[4] J. Callas, L. Donnerhacke, H. Finney, D. Shaw, và R. Thayer, “OpenPGP Message Format”, RFC 4880, 2007.

[5] P. Wouters và D. Huigens, “OpenPGP”, RFC 9580, 2024.

[6] X. Zhou và X. Tang, “Research and implementation of RSA algorithm for encryption and decryption”, trong *Proceedings of 2011 6th International Forum on Strategic Technology (IFOST)*, IEEE, 2011, tr 1118–1121.

[7] K. Moriarty, B. Kaliski, J. Jonsson, và A. Rusch, “PKCS #1: RSA Cryptography Specifications Version 2.2”, RFC 8017, 2016.

[8] D. Coppersmith, “Small Solutions to Polynomial Equations, and Low Exponent RSA Attacks”, *Journal of Cryptology*, tập 10, số 4, tr 233–260, 1997.

[9] D. A. McGrew và J. Viega, “The Security and Performance of the Galois/Counter Mode (GCM) of Operation”, trong *Fast Software Encryption (FSE 2004)*, trong Lecture Notes in Computer Science, vol. 3017. Springer, 2004, tr 343–355.

[10] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC”, NIST Special Publication 800–38D, 2007.

[11] W. Diffie và M. E. Hellman, “New Directions in Cryptography”, *IEEE Transactions on Information Theory*, tập 22, số 6, tr 644–654, 1976.

[12] M. O. Rabin, “Probabilistic algorithm for testing primality”, *Journal of Number Theory*, tập 12, số 1, tr 128–138, 1980.

PHỤ LỤC

Phụ lục A: Cài đặt giải thuật Euclid mở rộng và RSA số học

Mã nguồn mô-đun `algorithms/euclid.py`:

```python
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(e, phi):
    gcd, x, _ = extended_gcd(e, phi)
    if gcd != 1:
        raise ValueError(f"Không tồn tại nghịch đảo modulo: gcd({e}, {phi}) = {gcd} != 1")
    return (x % phi + phi) % phi
```

Mã nguồn mô-đun `algorithms/rsa.py`:

```python
import random
from .euclid import extended_gcd, mod_inverse

def is_prime(n, k=20):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2

    rng = random.SystemRandom()
    for _ in range(k):
        a = rng.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
```

```
        else:
            return False
    return True

def generate_prime(bits=128):
    rng = random.SystemRandom()
    while True:
        candidate = rng.getrandbits(bits) | (1 << (bits - 1)) | 1
        if is_prime(candidate):
            return candidate

def generate_rsa_keys(p=None, q=None, e=65537, bits=256):
    if p is None or q is None:
        prime_bits = max(bits // 2, 72)
        while True:
            p = generate_prime(prime_bits)
            q = generate_prime(prime_bits)
            if p != q and extended_gcd(e, (p - 1) * (q - 1))[0] == 1:
                break
    n = p * q
    phi = (p - 1) * (q - 1)
    d = mod_inverse(e, phi)
    return (e, n), (d, n)

def rsa_encrypt(public_key, plaintext_num):
    e, n = public_key
    if plaintext_num >= n:
        raise ValueError(f"Dữ liệu ({plaintext_num}) phải nhỏ hơn modulus n ({n})")
    return pow(plaintext_num, e, n)

def rsa_decrypt(private_key, ciphertext_num):
    d, n = private_key
    return pow(ciphertext_num, d, n)
```

Phụ lục B: Cài đặt AES-GCM và bộ đóng gói PGP Hybrid

Mã nguồn mô-đun `algorithms/aes_gcm.py` và `algorithms/hybrid_pgp.py`:

Python

```
import os
from Cryptodome.Cipher import AES
from .rsa import rsa_encrypt, rsa_decrypt

def aes_gcm_encrypt(session_key, plaintext_bytes):
    nonce = os.urandom(12)
    cipher = AES.new(session_key, AES.MODE_GCM, nonce=nonce)
    ciphertext, auth_tag = cipher.encrypt_and_digest(plaintext_bytes)
    return ciphertext, nonce, auth_tag

def aes_gcm_decrypt(session_key, nonce_bytes, ciphertext_bytes, auth_tag_bytes):
```

```
    cipher = AES.new(session_key, AES.MODE_GCM, nonce=nonce_bytes)
    return cipher.decrypt_and_verify(ciphertext_bytes, auth_tag_bytes)

# --- Đóng gói PGP Hybrid ---
def alice_encrypt_package(plaintext_str, bob_pub_key):
    session_key = os.urandom(16)  # Khóa phiên AES-128 (16 bytes = 128 bits)
    ciphertext, nonce, auth_tag = aes_gcm_encrypt(session_key,
    plaintext_str.encode("utf-8"))
    key_int = int.from_bytes(session_key, byteorder="big")
    enc_session_key = rsa_encrypt(bob_pub_key, key_int)
    package = {
        "enc_session_key": enc_session_key,
        "nonce": nonce,
        "ciphertext": ciphertext,
        "auth_tag": auth_tag,
    }
    return package, session_key

def bob_decrypt_package(package, bob_priv_key):
    recovered_key_int = rsa_decrypt(bob_priv_key, package["enc_session_key"])
    try:
        recovered_session_key = recovered_key_int.to_bytes(16, byteorder="big")
    except OverflowError:
        recovered_session_key = (recovered_key_int & ((1 << 128) - 1)).to_bytes(16,
        byteorder="big")
    plaintext_bytes = aes_gcm_decrypt(
        session_key=recovered_session_key,
        nonce_bytes=package["nonce"],
        ciphertext_bytes=package["ciphertext"],
        auth_tag_bytes=package["auth_tag"],
    )
    return plaintext_bytes.decode("utf-8")
```

Phụ lục C: Cài đặt truyền thông Socket TCP giữa 2 máy mạng LAN

Cơ chế đóng khung 4 byte độ dài giải quyết phân mảnh byte stream trong `PGP_RSA_AES_Socket_3_files/server_bob.py` :

Python

```
import struct, json

MAX_PACKET = 4 * 1024 * 1024

def recv_exact(connection, count):
    chunks = bytearray()
    while len(chunks) < count:
        chunk = connection.recv(min(count - len(chunks), 65536))
        if not chunk:
            raise ConnectionError("Bên gửi đã đóng kết nối trước khi gửi đủ dữ
            liệu.")
```

```
11          chunks.extend(chunk)
12      return bytes(chunks)
13
14  def send_json(connection, payload_dict):
15      data = json.dumps(payload_dict, separators=(",", ":")).encode("utf-8")
16      connection.sendall(struct.pack("!I", len(data)) + data)
17
18  def receive_json(connection):
19      packet_length = struct.unpack("!I", recv_exact(connection, 4))[0]
20      if not 0 < packet_length <= MAX_PACKET:
21          raise ValueError("Kích thước gói tin không hợp lệ.")
22      packet = recv_exact(connection, packet_length)
23      envelope = json.loads(packet.decode("utf-8"))
24      if not isinstance(envelope, dict):
25          raise ValueError("Gói tin phải là một JSON object.")
26      return envelope
```